

Werkzeugbaustein Matplotlib

Lukas Arnold	Simone Arnold	Florian Bagemihl
Matthias Baitsch	Marc Fehr	Franca Hollmann
Maik Poetzsch	Sebastian Seipel	

2026-05-26

Table of contents

Preamble	3
Intro	4
1 Einführung in Matplotlib	5
2 Diagrammtypen in Matplotlib	7
3 Anpassung und Gestaltung von Plots in Matplotlib	14
4 Erweiterte Techniken in Matplotlib	20
5 Best Practices in Matplotlib: Fehler und Verbesserungen	24
6 Das Wichtigste	33

Preamble



Bausteine Computergestützter Datenanalyse von Lukas Arnold, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch und Sebastian Seipel. “Werkzeugbaustein Matplotlib” von Marc Fehr ist lizenziert unter [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/). Das Werk ist abrufbar unter <https://github.com/bausteine-der-datenanalyse/w-python-matplotlib>. Ausgenommen von der Lizenz sind alle Logos und anders gekennzeichneten Inhalte. 2025

Zitiervorschlag

Arnold, Lukas, Simone Arnold, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch, und Sebastian Seipel. 2025. „Bausteine Computergestützter Datenanalyse. Werkzeugbaustein Matplotlib“. <https://github.com/bausteine-der-datenanalyse/w-python-matplotlib>.

BibTeX-Vorlage

```
@misc{BCD-python-matplotlib-2025,  
  title={Bausteine Computergestützter Datenanalyse. Werkzeugbaustein Matplotlib},  
  author={Arnold, Lukas and Arnold, Simone and Baitsch, Matthias and Fehr, Marc and Hollmann, Franca and Poetzsch, Maik and Seipel, Sebastian},  
  year={2025},  
  url={https://github.com/bausteine-der-datenanalyse/w-python-matplotlib}}
```

Intro

Voraussetzungen

- Grundlagen Python
- Einbinden von zusätzlichen Paketen

Verwendete Pakete und Datensätze

- matplotlib

Bearbeitungszeit

Geschätzte Bearbeitungszeit: 1h

Lernziele

- Einleitung: wie visualisiere ich Daten in Python
- Anpassen von Plots
- Do's & Dont's für wissenschaftliche Plots

1 Einführung in Matplotlib

Matplotlib ist eine der bekanntesten Bibliotheken zur Datenvisualisierung in Python. Sie ermöglicht das Erstellen statischer, animierter und interaktiver Diagramme mit hoher Flexibilität.

1.1 Warum Matplotlib?

- **Breite Unterstützung:** Funktioniert mit NumPy, Pandas und SciPy.
- **Hohe Anpassbarkeit:** Vollständige Kontrolle über Diagramme.
- **Integration in Jupyter Notebooks:** Ideal für interaktive Datenanalyse.
- **Kompatibilität:** Unterstützt verschiedene Ausgabeformate (PNG, SVG, PDF etc.).

1.2 Alternativen zu Matplotlib

Während Matplotlib leistungsstark ist, gibt es Alternativen, die für bestimmte Zwecke besser geeignet sein können: - **Seaborn:** Basiert auf Matplotlib, erleichtert statistische Visualisierung. - **Plotly:** Erzeugt interaktive Plots, gut für Dashboards. - **Bokeh:** Ideal für Web-Anwendungen mit interaktiven Visualisierungen.

1.3 Erstes Beispiel: Einfache Linie plotten

```
import matplotlib.pyplot as plt
import numpy as np

# Beispiel-Daten
t = np.linspace(0, 10, 100)
y = np.sin(t)

# Erstellen des Plots
```

```
plt.plot(t, y, label='sin(t)')
plt.xlabel('Zeit (s)')
plt.ylabel('Amplitude')
plt.title('Einfaches Linien-Diagramm')
plt.legend()
plt.show()
```

Dieses einfache Beispiel zeigt, wie man mit Matplotlib eine **Sinuskurve** visualisieren kann.

1.4 Nächste Schritte

Im nächsten Kapitel werden wir uns mit den verschiedenen Diagrammtypen beschäftigen, die Matplotlib bietet.

2 Diagrammtypen in Matplotlib

Matplotlib bietet eine Vielzahl von Diagrammtypen, die für unterschiedliche Zwecke geeignet sind. In diesem Kapitel werden die wichtigsten Diagrammtypen vorgestellt und ihre Anwendungsfälle erklärt.

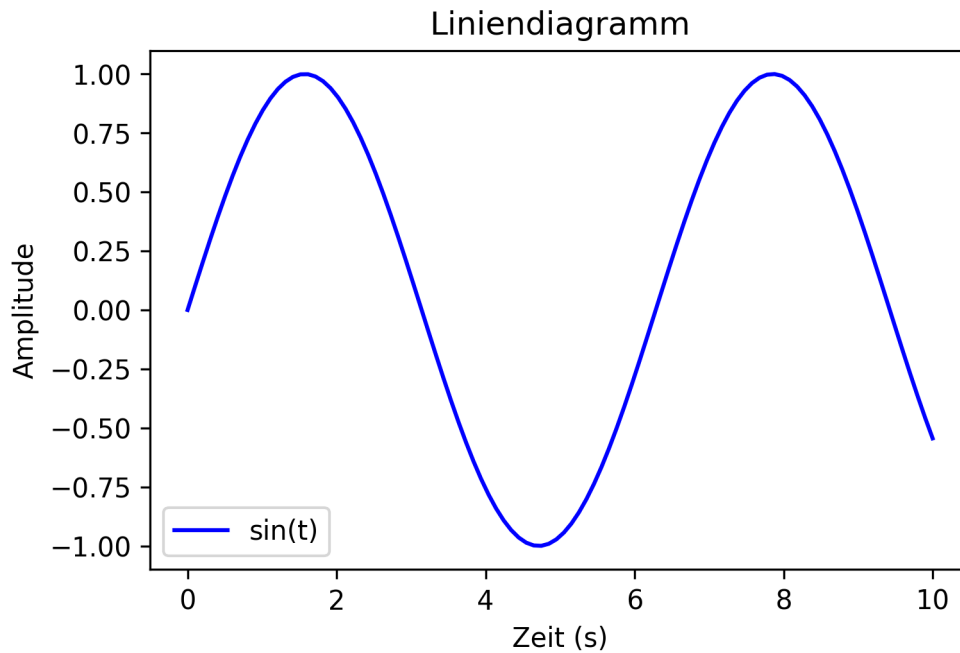
2.1 1. Liniendiagramme (`plt.plot()`)

Liniendiagramme eignen sich hervorragend zur Darstellung von Trends über Zeit.

```
import matplotlib.pyplot as plt
import numpy as np

t = np.linspace(0, 10, 100)
y = np.sin(t)

plt.plot(t, y, label='sin(t)', color='b')
plt.xlabel('Zeit (s)')
plt.ylabel('Amplitude')
plt.title('Liniendiagramm')
plt.legend()
plt.show()
```

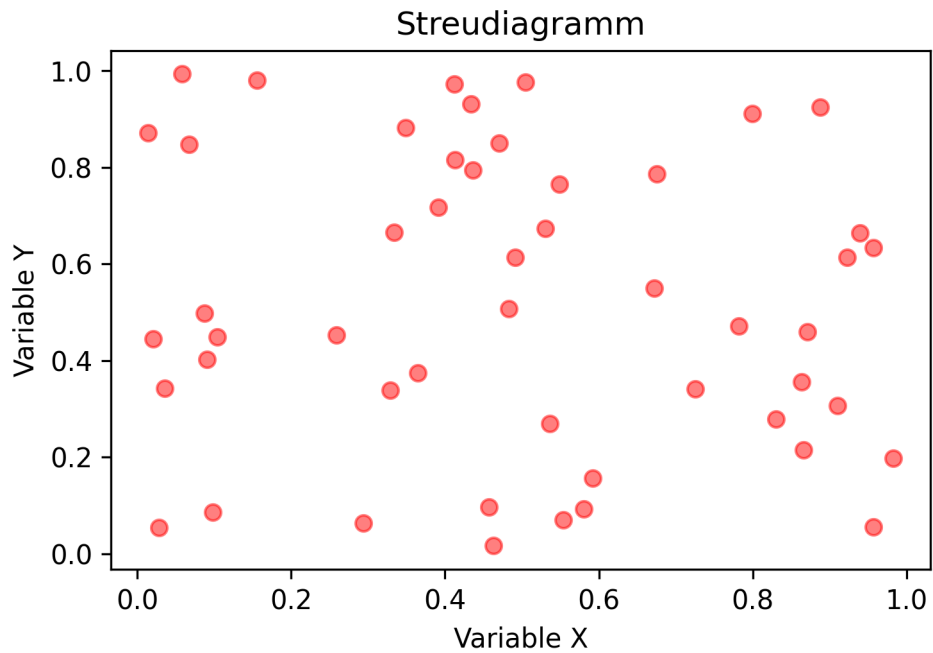


2.2 2. Streudiagramme (plt.scatter())

Streudiagramme werden verwendet, um Zusammenhänge zwischen zwei Variablen darzustellen.

```
x = np.random.rand(50)
y = np.random.rand(50)

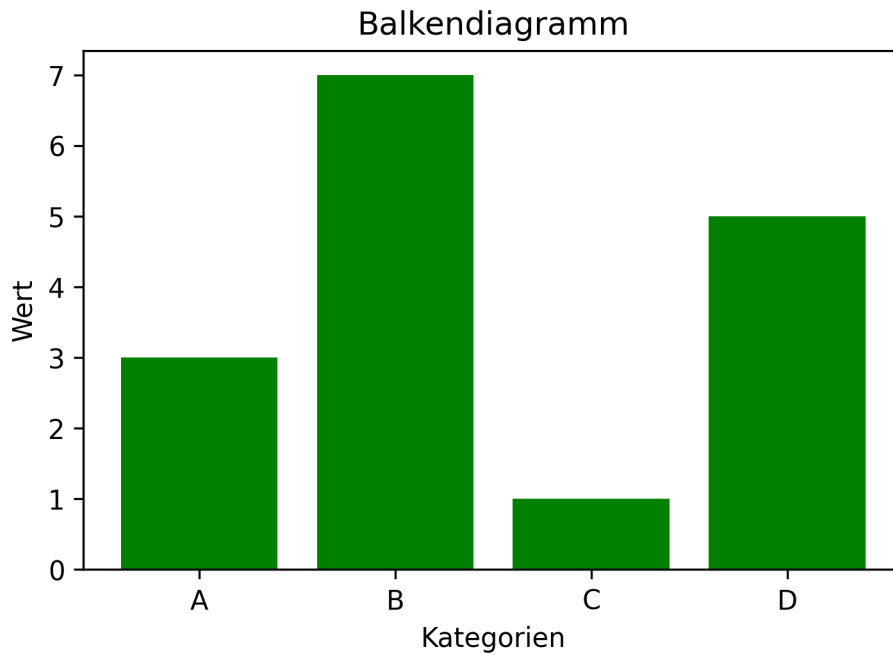
plt.scatter(x, y, color='r', alpha=0.5)
plt.xlabel('Variable X')
plt.ylabel('Variable Y')
plt.title('Streudiagramm')
plt.show()
```

2.3 3. Balkendiagramme (plt.bar())

Balkendiagramme eignen sich zur Darstellung kategorialer Daten.

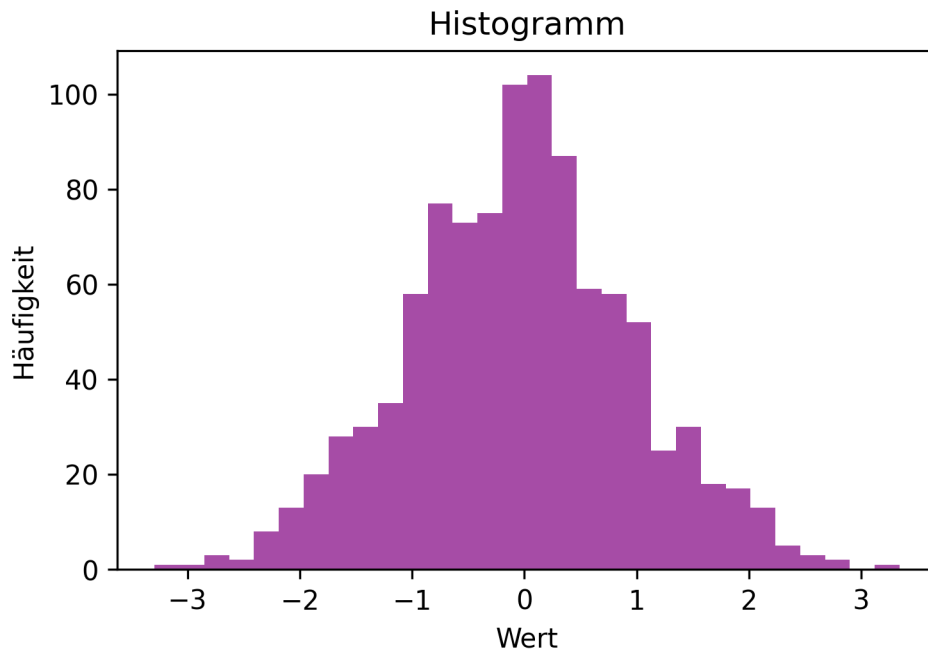
```
kategorien = ['A', 'B', 'C', 'D']  
werte = [3, 7, 1, 5]  
  
plt.bar(kategorien, werte, color='g')  
plt.xlabel('Kategorien')  
plt.ylabel('Wert')  
plt.title('Balkendiagramm')  
plt.show()
```



2.4 4. Histogramme (plt.hist())

Histogramme zeigen die Verteilung numerischer Daten.

```
daten = np.random.randn(1000)
plt.hist(daten, bins=30, color='purple', alpha=0.7)
plt.xlabel('Wert')
plt.ylabel('Häufigkeit')
plt.title('Histogramm')
plt.show()
```

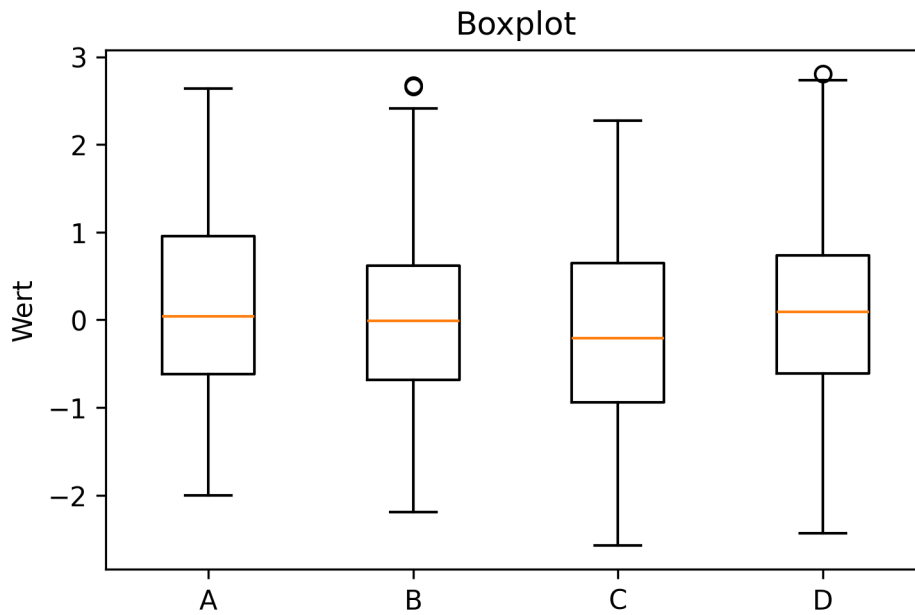


2.5 5. Boxplots (plt.boxplot())

Boxplots helfen, Ausreißer und die Verteilung von Daten zu visualisieren.

```
daten = [np.random.randn(100) for _ in range(4)]
plt.boxplot(daten, labels=['A', 'B', 'C', 'D'])
plt.ylabel('Wert')
plt.title('Boxplot')
plt.show()
```

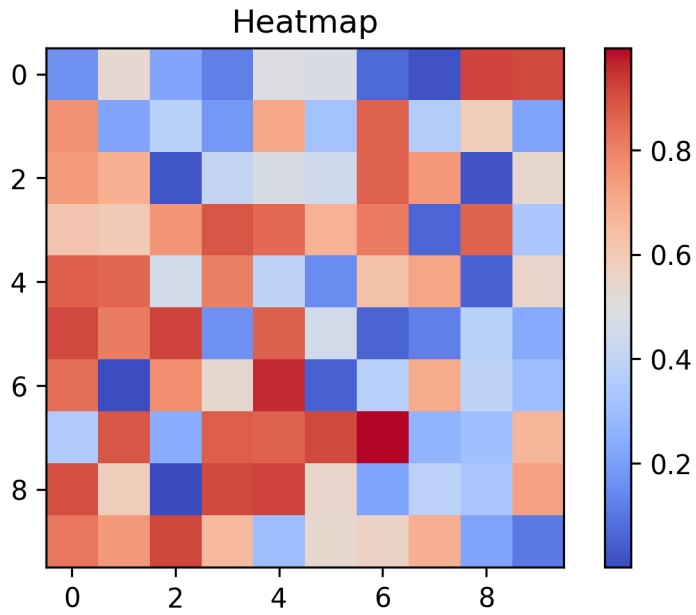
```
/var/folders/p_/ks3trxjx0jd839_g4g0vm4nc0000gn/T/ipykernel_7735/2728911591.py:2: MatplotlibD
plt.boxplot(daten, labels=['A', 'B', 'C', 'D'])
```



2.6 6. Heatmaps (plt.imshow())

Heatmaps eignen sich zur Darstellung von 2D-Daten.

```
daten = np.random.rand(10, 10)
plt.imshow(daten, cmap='coolwarm', interpolation='nearest')
plt.colorbar()
plt.title('Heatmap')
plt.show()
```



2.7 Fazit

Die Wahl des richtigen Diagrammtyps hängt von der Art der Daten und der gewünschten Darstellung ab. Im nächsten Kapitel werden wir uns mit der Anpassung und Gestaltung von Plots beschäftigen.

3 Anpassung und Gestaltung von Plots in Matplotlib

Ein gut gestaltetes Diagramm verbessert die Lesbarkeit und Verständlichkeit der dargestellten Daten. In diesem Kapitel werden wir verschiedene Möglichkeiten zur Anpassung und Gestaltung von Plots in Matplotlib erkunden.

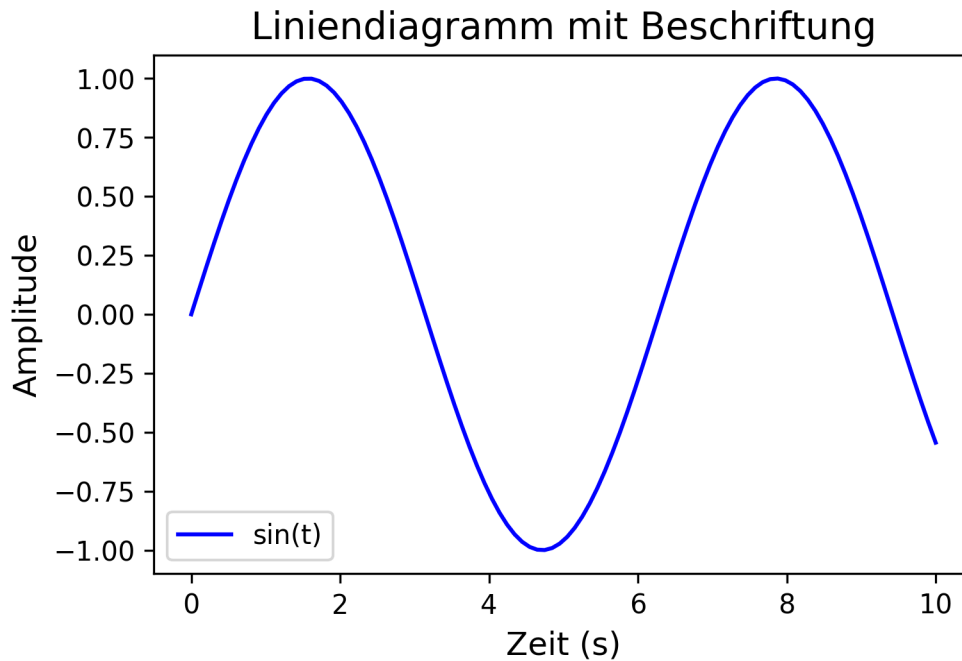
3.1 1. Achsentitel und Diagrammtitel

Klare Achsen- und Diagrammtitel sind essenziell für die Verständlichkeit eines Plots.

```
import matplotlib.pyplot as plt
import numpy as np

t = np.linspace(0, 10, 100)
y = np.sin(t)

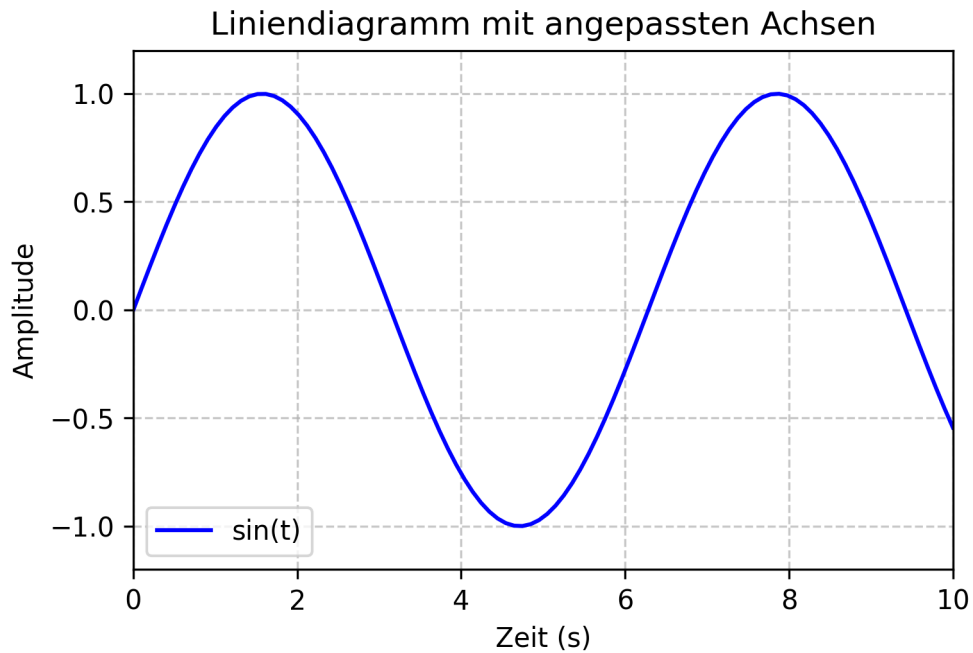
plt.plot(t, y, label='sin(t)', color='b')
plt.xlabel('Zeit (s)', fontsize=12)
plt.ylabel('Amplitude', fontsize=12)
plt.title('Liniendiagramm mit Beschriftung', fontsize=14)
plt.legend()
plt.show()
```



3.2 2. Anpassung der Achsen

Die Skalierung der Achsen sollte sinnvoll gewählt werden, um die Daten bestmöglich darzustellen.

```
plt.plot(t, y, label='sin(t)', color='b')
plt.xlabel('Zeit (s)')
plt.ylabel('Amplitude')
plt.xlim(0, 10)
plt.ylim(-1.2, 1.2)
plt.grid(True, linestyle='--', alpha=0.7)
plt.title('Liniendiagramm mit angepassten Achsen')
plt.legend()
plt.show()
```



3.3 3. Farben und Linienstile

Farben und Linienstile helfen dabei, wichtige Informationen im Plot hervorzuheben.

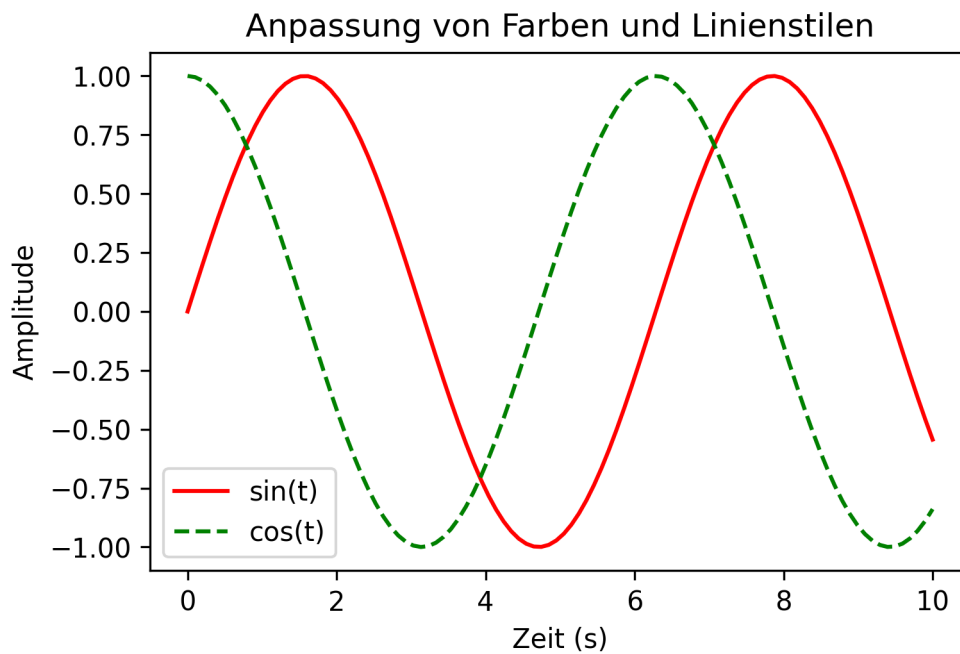
3.3.1 Wichtige Farben (Standardfarben in Matplotlib)

Farbe	Kürzel	Beschreibung
Blau	'b'	blue
Grün	'g'	green
Rot	'r'	red
Cyan	'c'	cyan
Magenta	'm'	magenta
Gelb	'y'	yellow
Schwarz	'k'	black
Weiß	'w'	white

3.3.2 Wichtige Linienstile

Linienstil	Kürzel	Beschreibung
Durchgezogen	'-'	Standardlinie
Gestrichelt	'_'	lange Striche
Gepunktet	':'	nur Punkte
Strich-Punkt	'-.'	abwechselnd Strich-Punkt

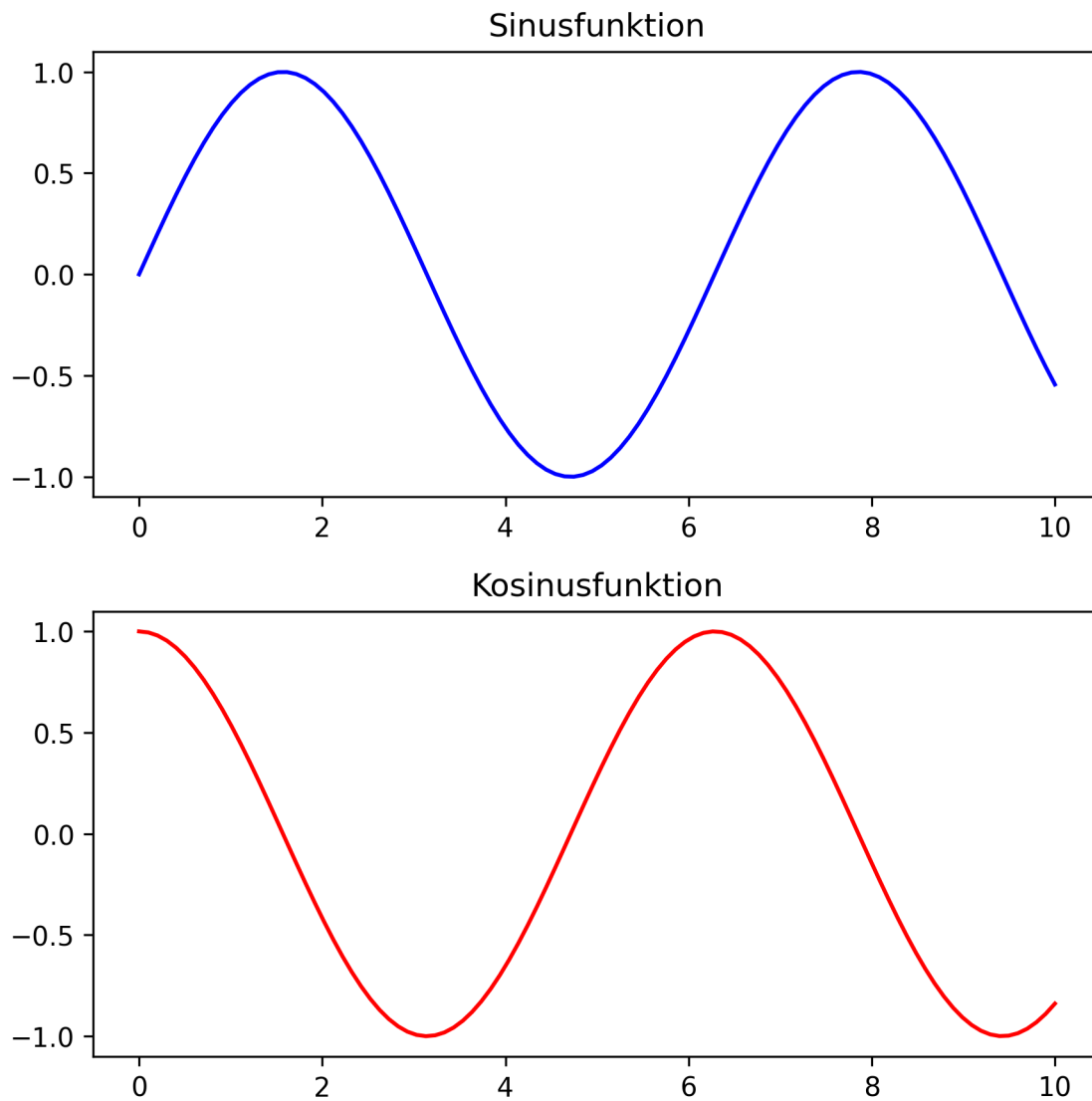
```
plt.plot(t, np.sin(t), linestyle='-', color='r', label='sin(t)')
plt.plot(t, np.cos(t), linestyle='--', color='g', label='cos(t)')
plt.xlabel('Zeit (s)')
plt.ylabel('Amplitude')
plt.title('Anpassung von Farben und Linienstilen')
plt.legend()
plt.show()
```



3.4 4. Mehrere Plots mit Subplots

Manchmal ist es sinnvoll, mehrere Diagramme in einer Abbildung darzustellen.

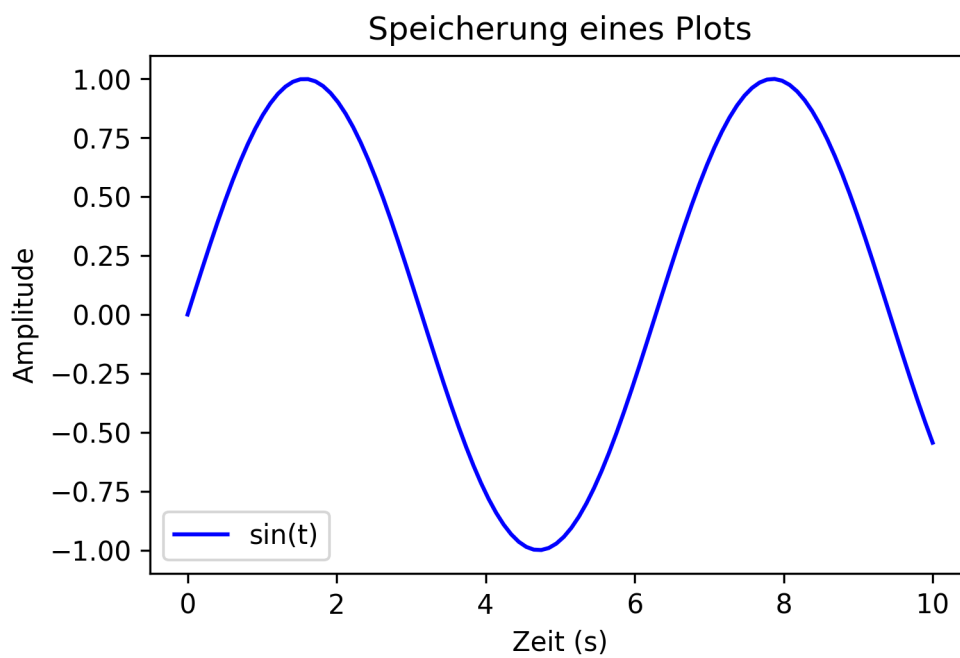
```
fig, axs = plt.subplots(2, 1, figsize=(6, 6))
axs[0].plot(t, np.sin(t), color='b')
axs[0].set_title('Sinusfunktion')
axs[1].plot(t, np.cos(t), color='r')
axs[1].set_title('Kosinusfunktion')
plt.tight_layout()
plt.show()
```



3.5 5. Speichern von Plots

Man kann Diagramme in verschiedenen Formaten speichern.

```
plt.plot(t, y, label='sin(t)', color='b')
plt.xlabel('Zeit (s)')
plt.ylabel('Amplitude')
plt.title('Speicherung eines Plots')
plt.legend()
plt.savefig('mein_plot.png', dpi=300)
plt.show()
```



3.6 Fazit

Durch geschickte Anpassungen lassen sich wissenschaftliche Plots deutlich verbessern. Im nächsten Kapitel werden wir uns mit erweiterten Techniken wie logarithmischen Skalen und Annotationen beschäftigen.

4 Erweiterte Techniken in Matplotlib

In diesem Kapitel betrachten wir einige fortgeschrittene Funktionen von Matplotlib, die für die wissenschaftliche Datenvisualisierung besonders nützlich sind.

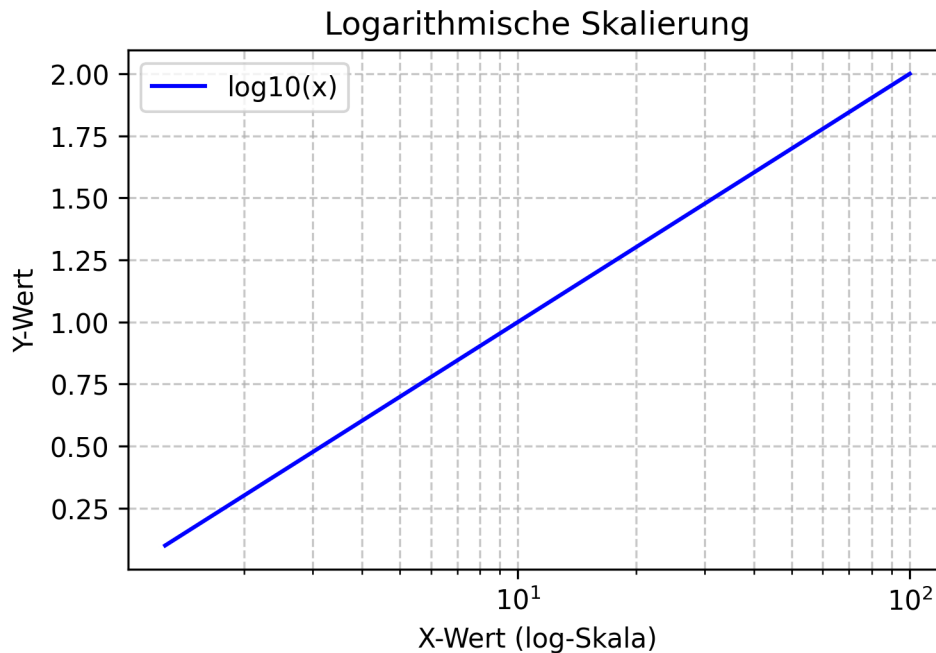
4.1 1. Logarithmische Skalen

Logarithmische Skalen werden oft verwendet, wenn Werte große Größenordnungen umfassen.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.logspace(0.1, 2, 100)
y = np.log10(x)

plt.plot(x, y, label='log10(x)', color='b')
plt.xscale('log')
plt.xlabel('X-Wert (log-Skala)')
plt.ylabel('Y-Wert')
plt.title('Logarithmische Skalierung')
plt.legend()
plt.grid(True, which='both', linestyle='--', alpha=0.7)
plt.show()
```



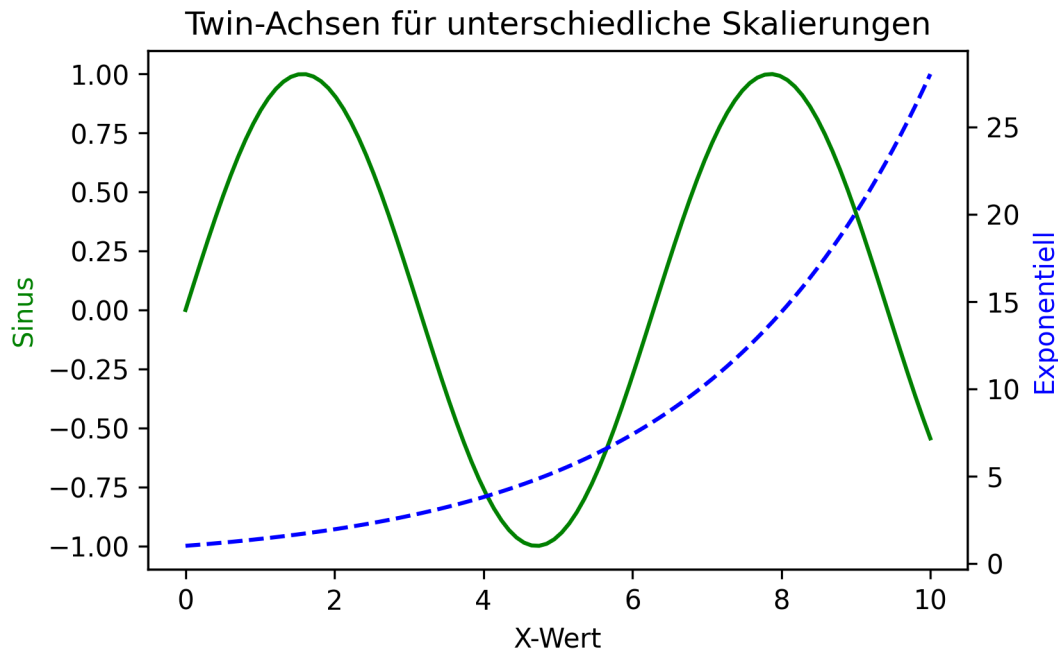
4.2 2. Twin-Achsen für verschiedene Skalierungen

Manchmal möchte man zwei verschiedene y-Achsen in einem Plot darstellen.

```
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.exp(x / 3)

fig, ax1 = plt.subplots()
ax2 = ax1.twinx()
ax1.plot(x, y1, 'g-', label='sin(x)')
ax2.plot(x, y2, 'b--', label='exp(x/3)')

ax1.set_xlabel('X-Wert')
ax1.set_ylabel('Sinus', color='g')
ax2.set_ylabel('Exponentiell', color='b')
ax1.set_title('Twin-Achsen für unterschiedliche Skalierungen')
plt.show()
```

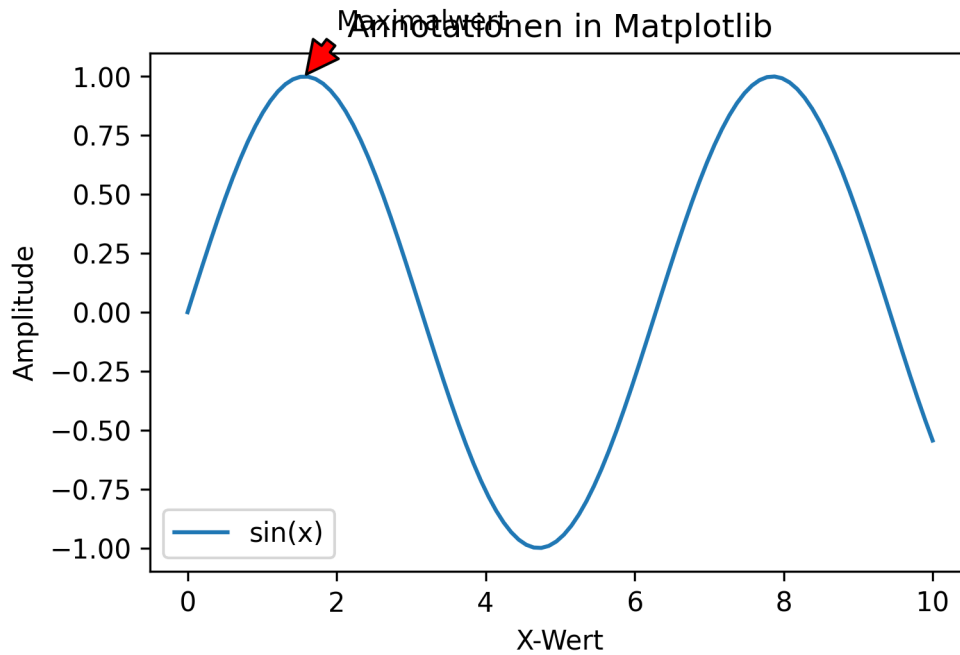


4.3 3. Annotationen in Diagrammen

Wichtige Punkte oder Werte in einem Diagramm können mit Annotationen hervorgehoben werden.

```
x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.plot(x, y, label='sin(x)')
plt.xlabel('X-Wert')
plt.ylabel('Amplitude')
plt.title('Annotationen in Matplotlib')
plt.annotate('Maximalwert', xy=(np.pi/2, 1), xytext=(2, 1.2),
            arrowprops=dict(facecolor='red', shrink=0.05))
plt.legend()
plt.show()
```



4.4 Fazit

Diese erweiterten Funktionen helfen dabei, wissenschaftliche Plots noch informativer zu gestalten. Im nächsten Kapitel werden wir Best Practices und typische Fehler in der wissenschaftlichen Visualisierung betrachten.

5 Best Practices in Matplotlib: Fehler und Verbesserungen

In diesem Kapitel zeigen wir für häufige Problemstellungen jeweils ein schlechtes und ein verbessertes Beispiel.

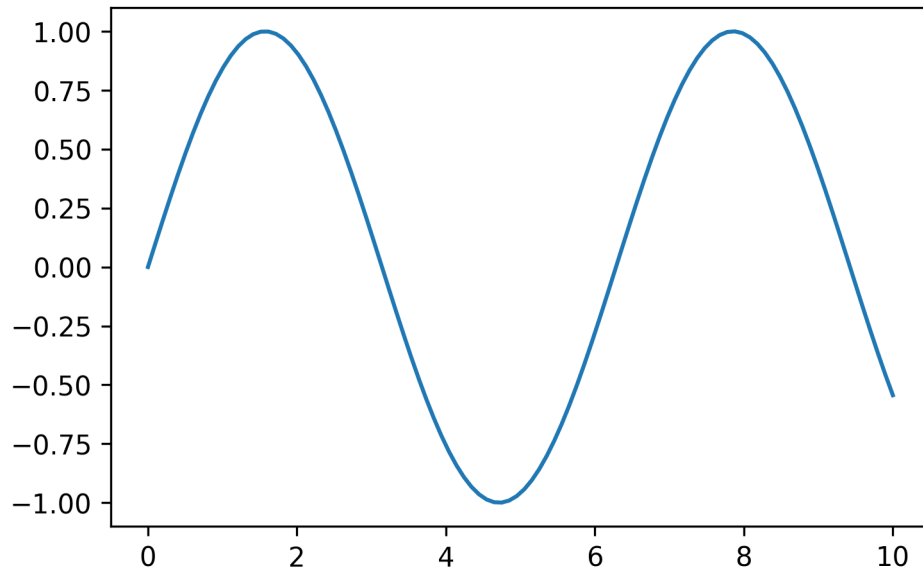
5.1 1. Fehlende Beschriftungen

5.1.1 Schlechtes Beispiel

```
import matplotlib.pyplot as plt
import numpy as np

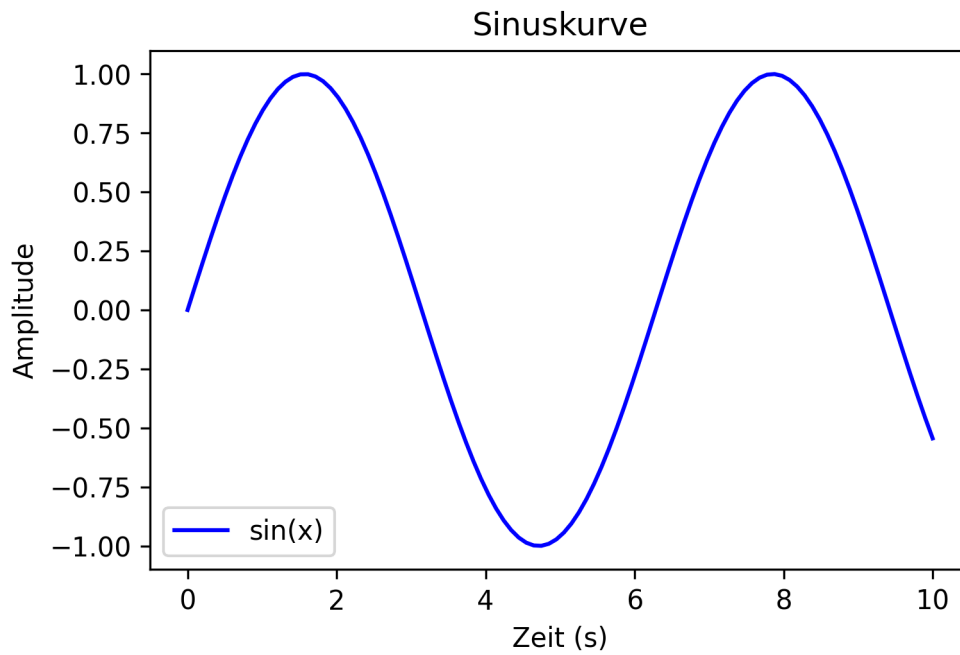
x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.plot(x, y)
plt.show()
```

5.1.2 Besseres Beispiel

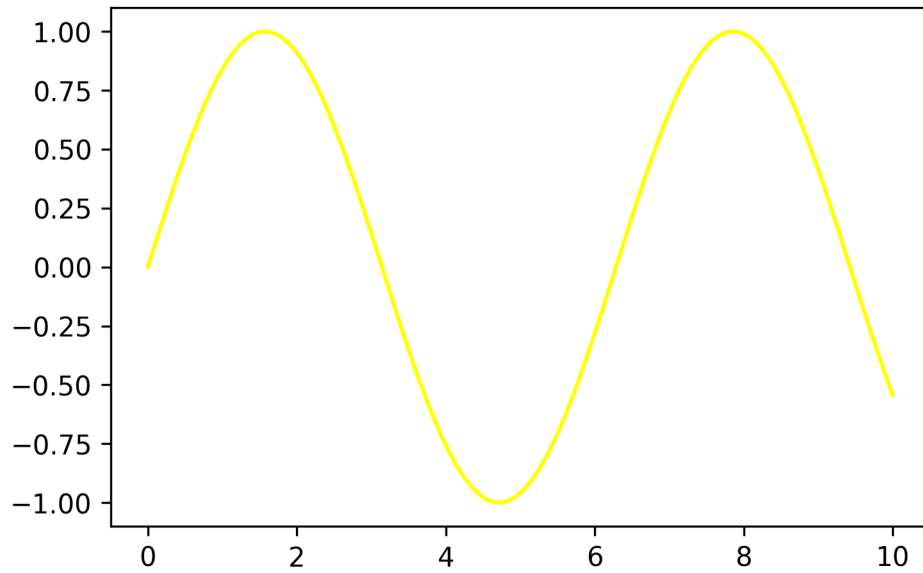
```
plt.plot(x, y, label='sin(x)', color='b')
plt.xlabel('Zeit (s)')
plt.ylabel('Amplitude')
plt.title('Sinuskurve')
plt.legend()
plt.show()
```



5.2 2. Ungünstige Farbwahl

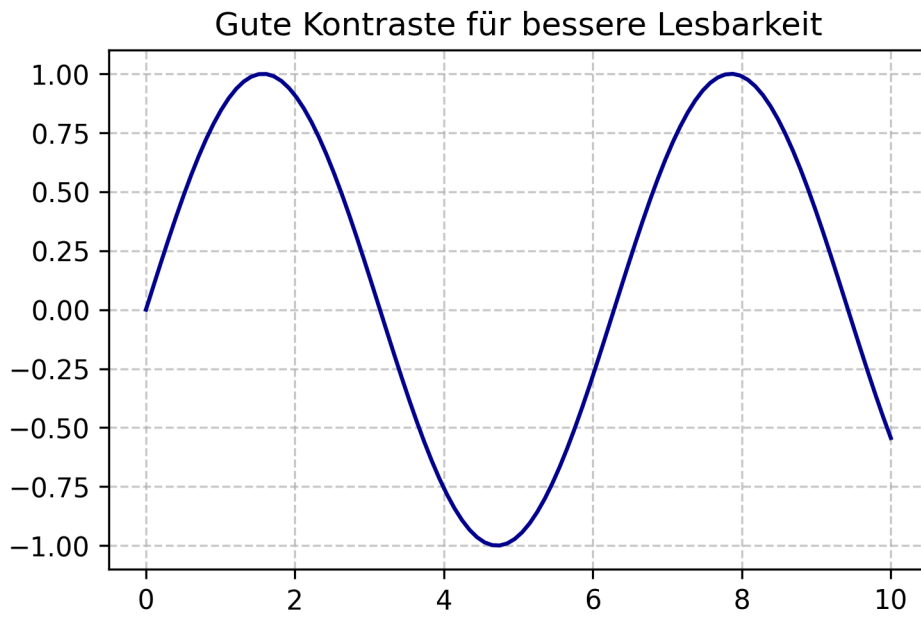
5.2.1 Schlechtes Beispiel

```
plt.plot(x, y, color='yellow')  
plt.show()
```



5.2.2 Besseres Beispiel

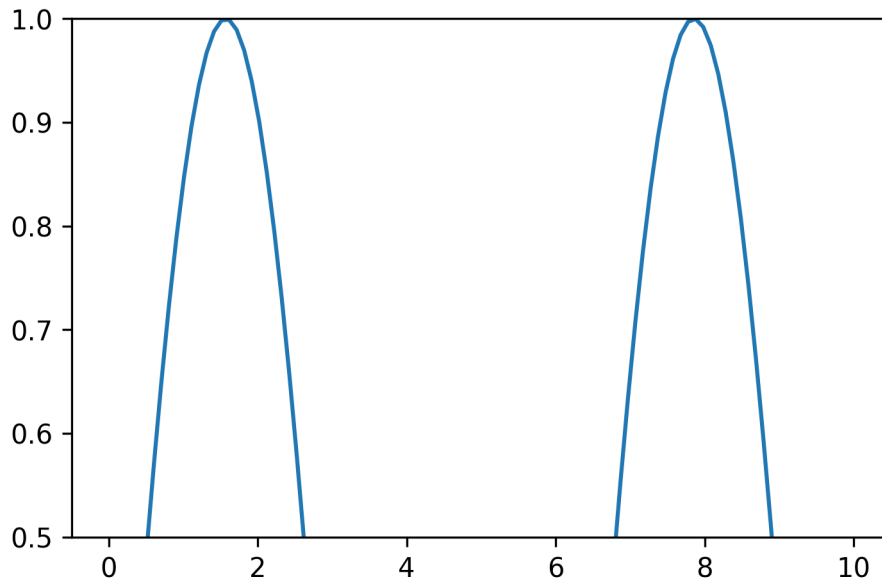
```
plt.plot(x, y, color='darkblue')
plt.grid(True, linestyle='--', alpha=0.7)
plt.title('Gute Kontraste für bessere Lesbarkeit')
plt.show()
```



5.3 3. Keine sinnvolle Achsenskalierung

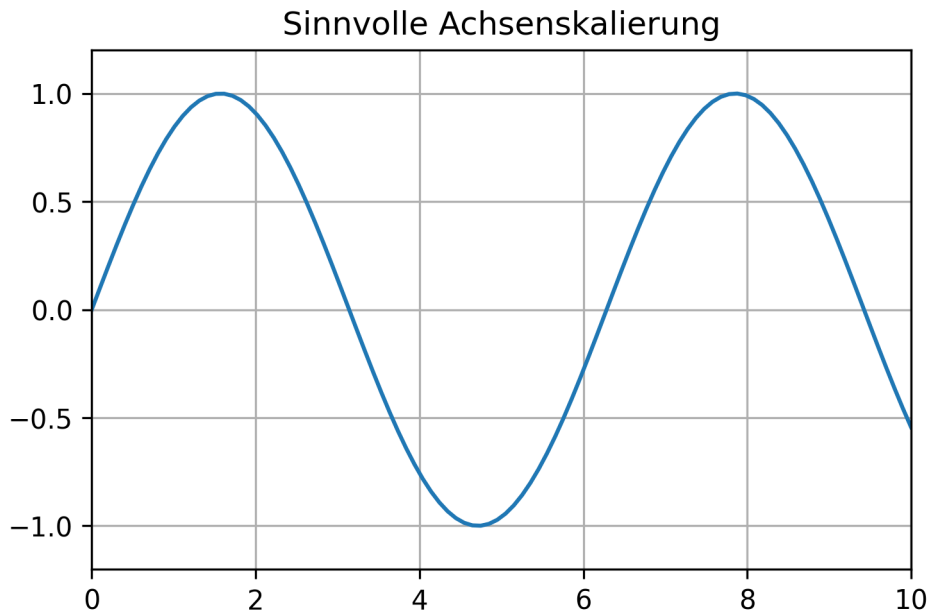
5.3.1 Schlechtes Beispiel

```
plt.plot(x, y)
plt.ylim(0.5, 1)
plt.show()
```



5.3.2 Besseres Beispiel

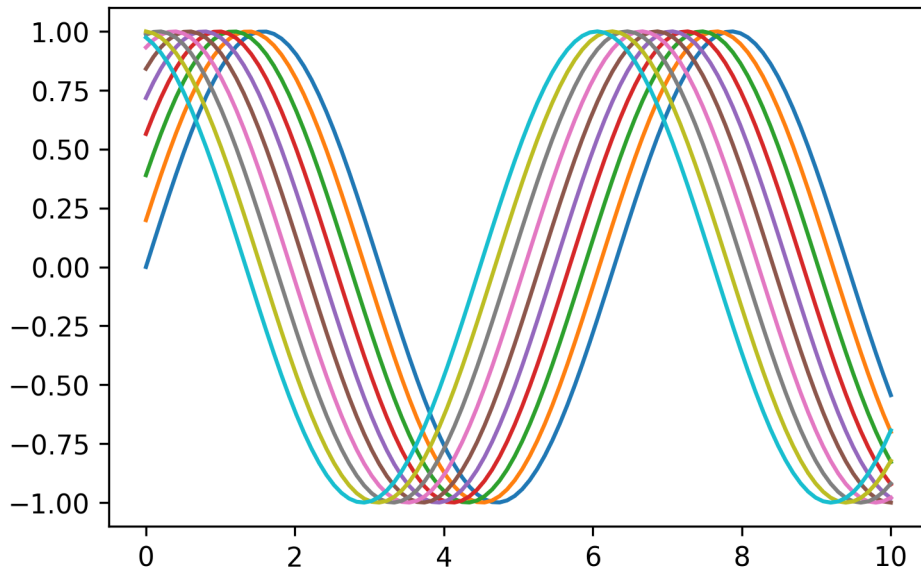
```
plt.plot(x, y)
plt.ylim(-1.2, 1.2)
plt.xlim(0, 10)
plt.grid(True)
plt.title('Sinnvolle Achsenskalierung')
plt.show()
```



5.4 4. Überladung durch zu viele Linien

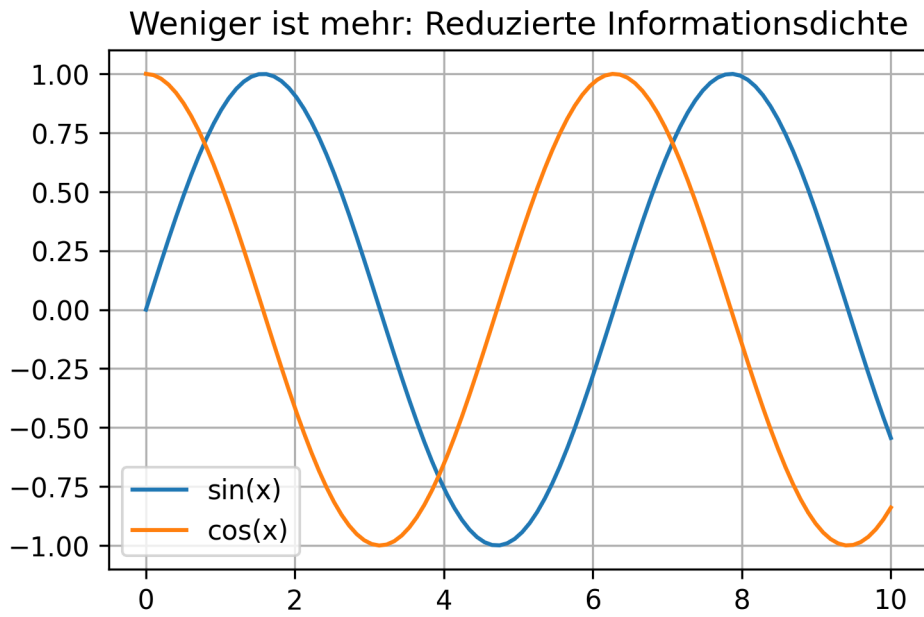
5.4.1 Schlechtes Beispiel

```
for i in range(10):  
    plt.plot(x, np.sin(x + i * 0.2))  
plt.show()
```



5.4.2 Besseres Beispiel

```
plt.plot(x, np.sin(x), label='sin(x)')
plt.plot(x, np.cos(x), label='cos(x)')
plt.legend()
plt.title('Weniger ist mehr: Reduzierte Informationsdichte')
plt.grid(True)
plt.show()
```



5.5 Fazit

Gute Plots zeichnen sich durch klare Beschriftungen, gute Lesbarkeit und eine sinnvolle Informationsdichte aus.

6 Das Wichtigste